

MODUL 5 REAL-TIME OPERATING SYSTEM

Nesa Dwi Cahyani (H1H024024)

Asisten: Muhammad Nur Bijak B

Tanggal Percobaan: 7/05/2026

TK244005-Nama Sistem Mikrokontroler

Laboratorium Multimedia – Jurusan Informatika UNSOED

Abstrak

Penggunaan Real-Time Operating System (RTOS) menjadi solusi penting dalam pengembangan embedded system yang membutuhkan pengelolaan banyak tugas secara efisien dan deterministik. Pada platform Arduino, penerapan RTOS memungkinkan perubahan dari pemrograman prosedural konvensional berbasis fungsi `loop()` tunggal menjadi sistem multitasking yang lebih kompleks. Praktikum ini bertujuan untuk memahami prinsip kerja RTOS, khususnya pada proses penjadwalan tugas (*task scheduling*) serta mekanisme komunikasi antar-tugas menggunakan kernel FreeRTOS.

Metode yang digunakan dalam penelitian ini meliputi pembuatan beberapa task yang berjalan secara konkuren menggunakan fungsi `xTaskCreate` serta penerapan protokol Queue untuk pertukaran data antar-tugas. Hasil pengujian menunjukkan bahwa meskipun dijalankan pada mikrokontroler dengan inti tunggal (*single-core*), RTOS mampu melakukan manajemen waktu melalui mekanisme *context switching* sehingga setiap tugas seolah-olah berjalan secara bersamaan. Selain itu, penggunaan Queue terbukti efektif dalam menjaga sinkronisasi data dan mencegah terjadinya *race condition* ketika dua atau lebih tugas mengakses sumber daya yang sama secara bersamaan.

Dengan demikian, implementasi RTOS pada mikrokontroler memberikan keunggulan dalam hal stabilitas serta waktu respons sistem yang lebih terprediksi. Hal ini menjadikan RTOS sangat relevan untuk diterapkan pada sistem yang kompleks, seperti robotika, sistem pemantauan multi-sensor, dan perangkat Internet of Things (IoT) yang membutuhkan performa real-time.

Kata kunci: RTOS, FreeRTOS, Arduino, Multitasking, Queue, Penjadwalan Task, Sistem Tertanam.

1. PENDAHULUAN

Perkembangan teknologi *embedded system* saat ini semakin menuntut perangkat untuk mampu menjalankan berbagai proses secara bersamaan

dengan tingkat ketepatan waktu yang tinggi. Pada pemrograman mikrokontroler konvensional, sistem umumnya menggunakan metode *super-loop* melalui fungsi `loop()` sehingga seluruh instruksi dijalankan secara berurutan. Metode ini cukup efektif untuk sistem sederhana, namun pada aplikasi yang lebih kompleks dapat menimbulkan keterlambatan respons (*latency*) karena setiap tugas harus menunggu tugas lainnya selesai dieksekusi. Oleh karena itu, penggunaan *Real-Time Operating System* (RTOS) menjadi solusi penting untuk mendukung pengelolaan banyak tugas secara efisien dan deterministik [1], [4].

Salah satu implementasi RTOS yang banyak digunakan pada sistem mikrokontroler adalah FreeRTOS. FreeRTOS merupakan kernel sistem operasi ringan yang mendukung mekanisme *multitasking* pada perangkat dengan sumber daya terbatas, termasuk Arduino Uno. Melalui mekanisme *task scheduling* dan *context switching*, FreeRTOS memungkinkan beberapa tugas dijalankan secara bergantian dalam waktu yang sangat cepat sehingga sistem tampak bekerja secara paralel [1], [5].

Selain mendukung *multitasking*, RTOS juga menyediakan pengaturan prioritas tugas sehingga proses yang lebih penting dapat diproses terlebih dahulu. Kemampuan ini sangat penting dalam sistem *real-time* karena beberapa aplikasi memerlukan respons dalam batas waktu tertentu (*deadline*). Kegagalan dalam memenuhi batas waktu tersebut dapat menyebabkan sistem tidak bekerja sebagaimana mestinya, terutama pada aplikasi kendali industri, robotika, maupun sistem pemantauan otomatis [2], [4].

Dalam lingkungan *multitasking*, komunikasi dan sinkronisasi antar-tugas menjadi aspek penting yang perlu diperhatikan. FreeRTOS menyediakan berbagai mekanisme komunikasi, salah satunya adalah *Queue*. *Queue* berfungsi sebagai media pertukaran data antar-tugas secara aman dan terstruktur sehingga dapat mengurangi risiko terjadinya *race condition* maupun konflik akses data. Dengan mekanisme ini, data dari satu tugas dapat dikirim dan diterima oleh tugas lain tanpa mengganggu proses eksekusi masing-masing [3], [5].

Pada praktikum Modul 5 ini, dilakukan implementasi FreeRTOS pada Arduino Uno melalui beberapa percobaan utama. Percobaan pertama berfokus pada pembuatan *multitasking* sederhana dengan mengendalikan beberapa LED menggunakan tugas yang berbeda serta waktu *delay* yang independen. Percobaan kedua memanfaatkan komunikasi antar-tugas menggunakan *Queue*, di mana satu tugas bertindak sebagai pengirim data, sedangkan tugas lainnya bertindak sebagai penerima dan pengolah data tersebut. Melalui percobaan ini, praktikan dapat memahami bagaimana RTOS mengatur penjadwalan tugas dan pertukaran data secara terstruktur.

Secara umum, praktikum ini bertujuan untuk memberikan pemahaman mengenai konsep dasar RTOS, mekanisme *multitasking*, penjadwalan tugas (*task scheduling*), serta komunikasi antar-tugas pada sistem tertanam berbasis Arduino Uno. Dengan demikian, praktikan diharapkan mampu memahami penerapan sistem operasi *real-time* dalam pengembangan *embedded system* yang lebih kompleks, responsif, dan andal [1], [2].

2. STUDI PUSTAKA

Pada praktikum ini, kajian teori difokuskan pada konsep *Real-Time Operating System* (RTOS) sebagai dasar dalam pengelolaan *multitasking* dan komunikasi antar-tugas pada sistem mikrokontroler. Selain itu, dibahas pula penerapan kernel FreeRTOS pada platform Arduino Uno untuk menangani beban kerja yang lebih kompleks secara deterministik.

2.1 KONSEP RTOS PADA SISTEM EMBEDDED

Dalam *embedded system*, kebutuhan terhadap respons yang cepat dan pengelolaan waktu yang presisi menjadi aspek yang sangat penting. Berbeda dengan sistem operasi umum (*General Purpose Operating System* atau GPOS) seperti Windows dan Linux, *Real-Time Operating System* (RTOS) dirancang untuk memberikan jaminan waktu respons terhadap suatu peristiwa eksternal. RTOS memungkinkan program dibagi menjadi beberapa tugas (*task*) yang dapat berjalan secara independen sehingga mendukung penerapan sistem *multitasking* yang lebih terstruktur [1], [4].

Penggunaan RTOS menjadi penting karena sistem tetap mampu bekerja secara responsif meskipun menjalankan beberapa proses secara bersamaan. Dengan adanya kernel RTOS, mikrokontroler tidak lagi hanya mengeksekusi instruksi secara berurutan dalam satu fungsi `loop()`, tetapi juga mampu mengatur penggunaan sumber daya prosesor berdasarkan

prioritas tugas yang sedang dijalankan. Melalui mekanisme *task scheduling*, CPU dapat dialihkan ke tugas dengan prioritas lebih tinggi ketika diperlukan sehingga efisiensi dan kestabilan sistem tetap terjaga [2], [5].

2.2 MANAJEMEN TASK DAN PENJADWALAN (SCHEDULING)

Manajemen tugas (*task management*) merupakan salah satu fitur utama dalam RTOS yang berfungsi untuk mengatur pelaksanaan beberapa tugas (*task*) pada satu prosesor. Pada mikrokontroler *single-core* seperti ATmega328P pada Arduino Uno, beberapa tugas tidak dijalankan secara paralel secara nyata, melainkan dieksekusi secara bergantian dalam interval waktu yang sangat cepat melalui mekanisme *context switching*. Proses pergantian tugas tersebut dikendalikan oleh *scheduler* atau penjadwal, yaitu bagian dari kernel RTOS yang menentukan tugas mana yang memperoleh akses CPU berdasarkan tingkat prioritasnya [1], [4].

Secara umum, RTOS dibedakan menjadi dua jenis, yaitu *Hard RTOS* dan *Soft RTOS*. *Hard RTOS* memberikan jaminan bahwa suatu tugas harus selesai sebelum batas waktu (*deadline*) yang telah ditentukan sehingga cocok digunakan pada sistem kritis. Sementara itu, *Soft RTOS* seperti FreeRTOS lebih fleksibel dalam pengelolaan waktu, tetapi tetap berupaya mempertahankan determinisme sistem agar respons tetap stabil dan terprediksi. Karakteristik tersebut memungkinkan pengembang memberikan prioritas yang lebih tinggi pada tugas penting, seperti pembacaan sensor darurat, dibandingkan tugas non kritis, misalnya pembaruan tampilan pada layar LCD [2], [5].

2.3 KOMUNIKASI TASK (QUEUE)

Komunikasi antar-tugas (*Inter-Task Communication*) merupakan mekanisme yang memungkinkan beberapa tugas (*task*) dalam RTOS saling bertukar data maupun melakukan sinkronisasi proses. Salah satu metode komunikasi yang paling umum digunakan adalah *Queue* atau antrian. *Queue* berfungsi sebagai media komunikasi yang aman (*thread-safe*), di mana satu tugas dapat mengirimkan data, sedangkan tugas lainnya menerima data tersebut secara teratur sesuai dengan urutan pengiriman [3], [5].

Dalam sistem *multitasking*, komunikasi data yang tidak terkontrol dapat menimbulkan *race condition*, yaitu kondisi ketika dua atau lebih tugas mencoba mengakses atau mengubah data yang sama secara bersamaan. Kondisi ini dapat menyebabkan ketidaksesuaian bahkan kerusakan

data dalam sistem. Oleh karena itu, penggunaan *Queue* pada FreeRTOS sangat penting untuk menjaga sinkronisasi aliran data antar-tugas sekaligus mencegah konflik penggunaan sumber daya memori pada mikrokontroler [3].

2.4 IMPLEMENTASI FREERTOS PADA ARDUINO UNO

FreeRTOS menyediakan berbagai *Application Programming Interface* (API) yang digunakan untuk mempermudah implementasi sistem RTOS pada Arduino Uno. Salah satu fungsi utama yang digunakan adalah `xTaskCreate()`, yang berfungsi untuk mendefinisikan serta membuat *task* baru di dalam sistem. Selain itu, terdapat fungsi `vTaskDelay()` yang digunakan untuk memberikan jeda eksekusi pada suatu *task* tanpa menghentikan kinerja kernel secara keseluruhan.

Berbeda dengan fungsi `delay()` pada Arduino standar yang bersifat *blocking* (menghentikan seluruh proses eksekusi program), `vTaskDelay()` bekerja dengan cara memindahkan *task* ke kondisi *Blocked*. Pada kondisi ini, CPU tidak berhenti bekerja, melainkan *scheduler* akan memberikan kesempatan eksekusi kepada *task* lain yang berada pada status *Ready*. Mekanisme ini memungkinkan sistem tetap berjalan secara *multitasking* tanpa mengganggu tugas lainnya [1], [5].

Dengan adanya integrasi FreeRTOS, Arduino Uno mampu menangani aplikasi yang lebih kompleks, seperti sistem pemantauan multi-sensor yang berjalan secara paralel. Hal ini meningkatkan efisiensi pemrosesan data serta membuat sistem lebih responsif terhadap perubahan kondisi secara *real-time* [1], [5].

2.5 APLIKASI RTOS DALAM SISTEM REAL-TIME

Penggunaan RTOS banyak diterapkan pada sistem cerdas yang membutuhkan tingkat keandalan tinggi, seperti kontrol robotika, sistem kendali otomotif, serta perangkat *Internet of Things* (IoT). Salah satu keunggulan utama RTOS adalah skalabilitasnya, di mana pengembang dapat menambahkan fungsi baru sebagai *task* tambahan tanpa mengganggu struktur program utama yang telah ada [2], [3].

Integrasi antara manajemen tugas (*task management*) dan komunikasi data yang terstruktur melalui *Queue* memungkinkan pengembangan sistem yang lebih adaptif dan efisien. Dengan demikian, pemahaman mengenai prinsip kerja RTOS menjadi kompetensi dasar yang sangat penting bagi pengembang sistem

tertanam untuk menciptakan solusi teknologi yang responsif dan andal (*reliable*) [1], [4].

3. METODOLOGI

Bagian metodologi ini menjelaskan secara sistematis mengenai alat, bahan, serta tahapan kerja yang dilakukan selama praktikum untuk mencapai tujuan yang telah ditetapkan pada Modul 5. Praktikum ini berfokus pada pemahaman prinsip kerja *Real-Time Operating System* (RTOS) serta penerapan *multitasking* dan komunikasi antar-*task* menggunakan mekanisme *Queue* pada kernel FreeRTOS di mikrokontroler Arduino Uno.

3.1 KOMPONEN DAN ALAT

Dalam praktikum ini digunakan perangkat keras yang dirakit pada *breadboard* guna mendukung pengujian sistem *multitasking* dan komunikasi data antar-komponen secara *real-time* [1], [3].

Tabel 3-1 Daftar Alat dan Bahan Praktikum

Nama Alat/Bahan	Jumlah	Fungsi
Arduino Uno R3	1 Unit	Unit kendali utama (mikrokontroler)
Breadboard	1 Unit	Papan penghubung rangkaian sementara
Potensiometer	1 Buah	Perangkat input analog untuk simulasi data
Sensor DHT	1 Buah	Perangkat input untuk menghasilkan data sensor dinamis pada percobaan komunikasi task
Kabel Jumper	Secukupnya	Penghubung elektrikal antar komponen
Kabel USB Type B	1 Buah	Komunikasi data dan catu daya dari PC
LED	2 Buah	Indikator output digital (protokol UART)
Resistor 220 Ohm	Secukupnya	Pembatas arus untuk melindungi LED/Seven Segment

3.2 PROSEDUR PERCOBAAN 5A: MULTITASKING

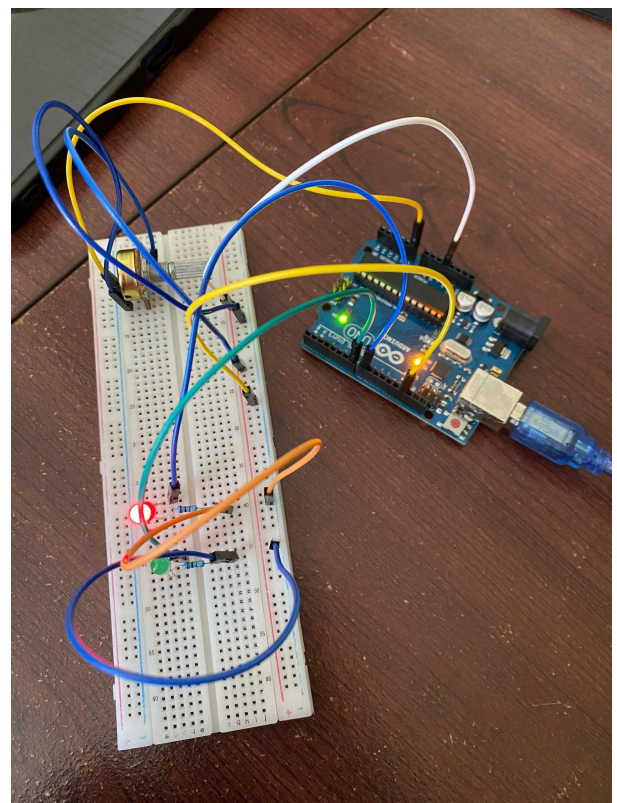
Percobaan 5A bertujuan untuk memahami konsep *multitasking* pada *embedded system* menggunakan FreeRTOS di Arduino Uno serta menjalankan beberapa *task* secara konkuren (*concurrent*). Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Langkah pertama yaitu menyiapkan perangkat keras yang digunakan, meliputi satu unit papan Arduino Uno R3, sebuah *breadboard*, dua buah LED (merah dan kuning), dua buah resistor pembatas arus (220 Ω atau 330 Ω), kabel USB Type-B, serta kabel jumper secukupnya.
2. Penyusunan rangkaian dilakukan pada *breadboard* sesuai dengan panduan skematik. Kaki anoda LED merah dihubungkan ke pin digital D10 Arduino melalui kabel jumper, sedangkan anoda LED kuning dihubungkan ke pin digital D8. Masing-masing kaki katoda LED dihubungkan secara seri dengan resistor pembatas arus menuju jalur *Ground* (GND) pada *breadboard* yang telah terintegrasi dengan pin GND Arduino Uno.
3. Pemrograman dilakukan pada Arduino IDE dengan mengimpor pustaka utama menggunakan perintah `#include <Arduino_FreeRTOS.h>`. Di dalam fungsi `setup()`, komunikasi serial diaktifkan pada kecepatan 9600 bps melalui perintah `Serial.begin(9600)`. Selanjutnya, program menginisialisasi tiga buah fungsi *task* independen menggunakan perintah `xTaskCreate()`. Ketiga *task* tersebut dialokasikan dengan ukuran *stack* sebesar 128 *word* dan diatur pada tingkat prioritas yang sama, yaitu prioritas 1. Kernel RTOS kemudian dijalankan secara resmi melalui perintah `vTaskStartScheduler()` pada akhir fungsi `setup()`, sedangkan fungsi utama `loop()` dibiarkan kosong.
4. Di luar fungsi utama, dibuat tiga logika subprogram yang berjalan di dalam perulangan tak terbatas (*while(1)*) sebagai berikut:
 - *TaskBlink1* dikonfigurasi untuk menyalakan dan mematikan LED merah pada pin D10 dengan jeda waktu *non-blocking* menggunakan fungsi `vTaskDelay(200/portTICK_PERIOD_MS)`.
 - *TaskBlink2* dikonfigurasi untuk menyalakan dan mematikan LED kuning pada pin D8 dengan jeda waktu `vTaskDelay(300/portTICK_PERIOD_MS)`.
 - *TaskPrint* diprogram sebagai pencatat data (*counter*) yang nilainya akan terus bertambah pada setiap siklus, kemudian

mengirimkan hasilnya ke komputer melalui perintah `Serial.println(counter)` setiap selang waktu `vTaskDelay(500/portTICK_PERIOD_MS)`.

5. Berkas program disimpan dengan nama *modul5_multitasking*. Pengujian dilakukan dengan melakukan verifikasi kode (*compile*) terlebih dahulu pada Arduino IDE untuk mendeteksi kesalahan sintaks. Setelah dipastikan tidak terdapat galat (*error*), program kemudian diunggah (*upload*) ke papan Arduino Uno melalui kabel USB Type-B.
6. Tahap terakhir dilakukan dengan membuka fitur *Serial Monitor* pada Arduino IDE. Praktikan mengamati pola kedipan kedua LED pada *breadboard* serta data numerik *counter* yang tercetak secara berkala pada layar monitor. Hasil pengamatan tersebut kemudian dianalisis untuk mengetahui apakah *scheduler* FreeRTOS berhasil membagi waktu eksekusi ketiga *task* secara konkuren (*multitasking*).

Representasi program akan berjalan seperti yang digambarkan pada diagram berikut.



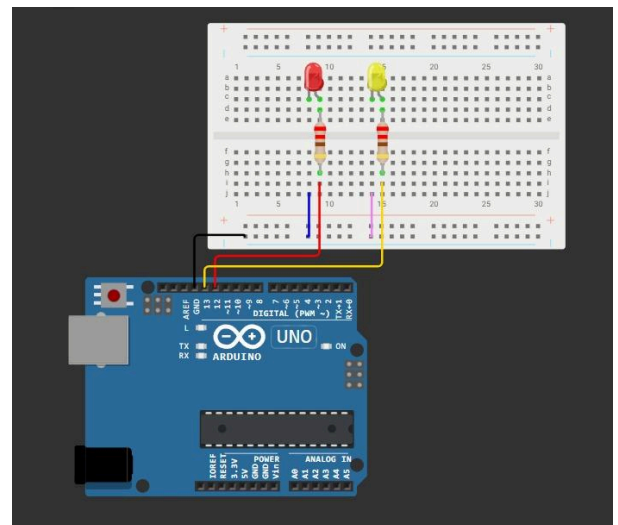
Gambar 3-1 Diagram Multitasking

3.3 PROSEDUR PERCOBAAN 5B: KOMUNIKASI TASK

Percobaan 5B bertujuan untuk memahami konsep *multitasking* serta mekanisme komunikasi antar-tugas (*inter-task communication*) menggunakan fitur *Queue* pada FreeRTOS di Arduino Uno. Adapun langkah-langkah yang dilakukan adalah sebagai berikut:

1. Langkah pertama adalah menyiapkan perangkat yang diperlukan, yaitu satu unit papan Arduino Uno, kabel USB Type-B, serta komputer yang telah terpasang Arduino IDE. Pada tahap awal percobaan ini, fokus utama diarahkan pada penggunaan *Serial Monitor* sebagai media *debugger* dan keluaran (*output*) sistem.
2. Papan Arduino Uno dihubungkan ke komputer menggunakan kabel USB. Karena percobaan awal ini menggunakan data statis, tidak diperlukan rangkaian sensor eksternal pada *breadboard*, sehingga koneksi utama hanya melibatkan komunikasi data antara Arduino dan komputer.
3. Pada Arduino IDE, pustaka yang diperlukan disertakan menggunakan perintah `#include <Arduino_FreeRTOS.h>` untuk kernel RTOS dan `#include <queue.h>` untuk manajemen antrian data. Selanjutnya, didefinisikan struktur data *readings* yang digunakan untuk menampung informasi suhu dan kelembapan, serta deklarasi variabel `QueueHandle_t my_queue` sebagai *handle* untuk antrian (*queue*) tersebut.
4. Di dalam fungsi `setup()`, komunikasi serial diaktifkan pada kecepatan 9600 bps. Selanjutnya, dibuat objek *queue* menggunakan perintah `xQueueCreate(1, sizeof(struct readings))` untuk menyiapkan ruang penyimpanan pesan. Langkah berikutnya adalah membuat dua buah *task* menggunakan fungsi `xTaskCreate()`, yaitu *task read_data* yang bertugas mengirimkan informasi dan *task display* yang berfungsi menampilkan informasi tersebut.
5. Pada fungsi `read_data`, dibuat perulangan tak terbatas yang bertugas mengisi variabel struktur dengan data statis, misalnya suhu 54 dan kelembapan 30. Data tersebut kemudian dikirim ke dalam antrian menggunakan fungsi `xQueueSend()`. Selanjutnya, ditambahkan fungsi `vTaskDelay(100)` untuk memberikan jeda waktu sebelum proses pengiriman data berikutnya dilakukan.
6. Pada fungsi `display`, program akan terus memantau antrian menggunakan fungsi `xQueueReceive()`. Jika data berhasil diterima (`pdPASS`), program akan menampilkan nilai suhu dan kelembapan ke *Serial Monitor* menggunakan perintah `Serial.print()` dan `Serial.println()`.
7. Lakukan verifikasi (*compile*) pada program untuk memastikan tidak terdapat kesalahan penulisan kode. Setelah program dipastikan benar, simpan berkas dengan nama `modul6_taskqueue`, kemudian unggah (*upload*) program tersebut ke papan Arduino Uno.
8. Buka fitur *Serial Monitor* pada Arduino IDE dan amati data yang ditampilkan. Perhatikan apakah nilai suhu dan kelembapan muncul secara berkala sesuai dengan spesifikasi yang diharapkan. Hasil pengamatan tersebut kemudian dianalisis untuk memahami bagaimana *scheduler* mengelola komunikasi antar-task dalam sistem tersebut.

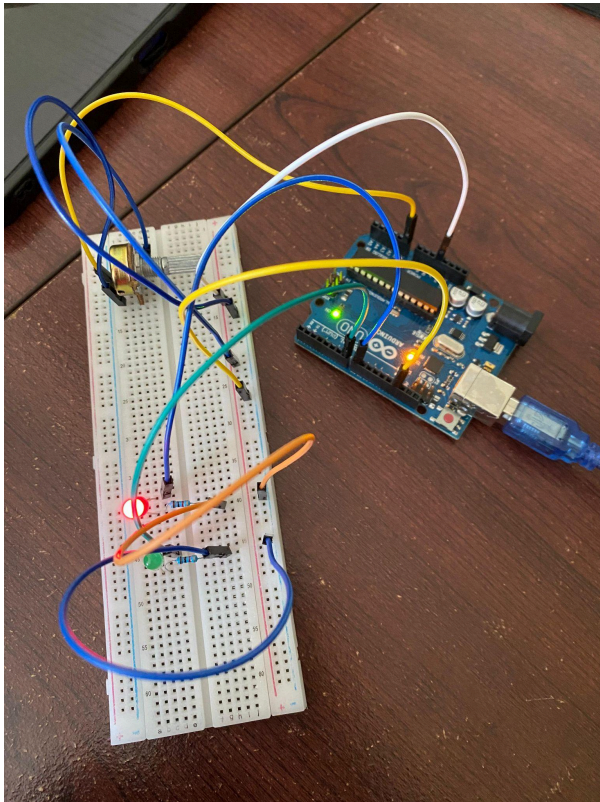
Representasi program akan berjalan seperti yang digambarkan pada diagram berikut.



Gambar 3-2 Diagram Komunikasi Task

4. HASIL DAN ANALISIS

4.1 ANALISIS HASIL PERCOBAAN



Gambar 4-1 Rangkaian Percobaan 1

Berdasarkan hasil pengamatan pada Percobaan 5A, rangkaian yang terdiri atas Arduino Uno dan dua buah LED menunjukkan bahwa implementasi *Real-Time Operating System* (RTOS) mampu meningkatkan efisiensi mikrokontroler dalam mengeksekusi beberapa fungsi secara bersamaan. Meskipun Arduino Uno merupakan perangkat berbasis inti tunggal (*single-core*), penggunaan kernel FreeRTOS berhasil mengatur pembagian waktu eksekusi CPU sehingga *TaskBlink1*, *TaskBlink2*, dan *TaskPrint* tampak berjalan secara konkuren. Hal ini menunjukkan bahwa mekanisme manajemen tugas (*task management*) pada RTOS telah berjalan dengan baik, di mana setiap instruksi dapat dialokasikan dan dieksekusi dengan tingkat keterlambatan (*latency*) yang sangat kecil [1], [4].

Keberhasilan simulasi *multitasking* ini terlihat dari pola kedipan kedua LED yang bekerja secara independen tanpa saling mengganggu. *TaskBlink1* yang mengendalikan pin 8 bekerja dengan interval 200 ms, sedangkan *TaskBlink2* pada pin 7 bekerja dengan interval 300 ms. Penggunaan fungsi `vTaskDelay()` menjadi faktor utama yang memungkinkan *scheduler* bekerja secara efektif dengan cara memindahkan *task* ke kondisi *Blocked* sehingga sumber daya CPU dapat dialihkan kepada *task* lain yang berada pada status *Ready*. Hal ini membuktikan bahwa Arduino Uno mampu menjalankan manajemen waktu yang lebih efisien dan deterministik dibandingkan

penggunaan fungsi `delay()` konvensional yang bersifat *blocking* [1], [5].

Selama pengujian melalui *Serial Monitor*, keluaran berupa teks "Task1", "Task2", dan nilai *counter* muncul secara teratur sesuai dengan waktu penjadwalan yang telah ditentukan. Kondisi tersebut menunjukkan bahwa kernel FreeRTOS mampu melakukan manajemen memori dan alokasi ukuran *stack* secara stabil untuk setiap *task* yang dibuat melalui fungsi `xTaskCreate()`. Stabilitas ini sangat penting karena pada sistem *real-time*, ketepatan waktu eksekusi menentukan kemampuan sistem dalam memenuhi batas waktu (*deadline*) agar tidak terjadi kegagalan fungsi [2], [4].

Secara keseluruhan, hasil percobaan ini menunjukkan bahwa integrasi RTOS pada platform Arduino Uno mampu mengubah pola pemrograman sekuensial pada struktur *loop()* menjadi sistem pemrograman berbasis *task* yang lebih paralel. Mekanisme tersebut mendukung pengembangan sistem tertanam yang lebih responsif, tangguh, dan andal dalam memproses masukan maupun keluaran secara bersamaan. Selain itu, penerapan *multitasking* ini menjadi dasar penting dalam pengembangan aplikasi yang lebih kompleks, seperti sistem otomasi industri, robotika, dan perangkat *Internet of Things* (IoT) yang memerlukan penanganan multi-sensor secara waktu nyata [2], [3].

4.1.1 JAWABAN PERTANYAAN PRAKTIKUM PERCOBAAN 1

1. Jelaskan perbedaan antara `loop()` pada Arduino biasa dengan sistem yang menggunakan RTOS!

Pada Arduino standar, fungsi `loop()` bekerja secara sekuensial (berurutan) dan bersifat *bare-metal*. Artinya, mikrokontroler hanya dapat menjalankan satu instruksi pada satu waktu. Apabila terdapat fungsi penundaan seperti `delay()`, maka seluruh proses eksekusi program akan berhenti hingga waktu tunda tersebut selesai.

Sebaliknya, pada sistem yang menggunakan *Real-Time Operating System* (RTOS), eksekusi program tidak bergantung pada satu *loop()* tunggal, melainkan dibagi menjadi beberapa *task* yang berjalan secara independen. RTOS memanfaatkan *scheduler* untuk mengatur pembagian waktu CPU antar-*task* berdasarkan tingkat prioritasnya. Mekanisme ini memungkinkan terjadinya *preemptive multitasking*, yaitu kondisi ketika *task* dengan prioritas lebih tinggi dapat menyela eksekusi *task* dengan prioritas lebih

rendah guna memastikan respons sistem tetap bersifat *real-time* [4], [5].

2. Mengapa fungsi `loop()` dibiarkan kosong?

Dalam implementasi Arduino yang menggunakan RTOS seperti FreeRTOS, fungsi `loop()` sering kali dibiarkan kosong karena seluruh logika utama program telah dipindahkan ke dalam beberapa *task* yang dibuat di dalam fungsi `setup()`.

Setelah *scheduler* RTOS dijalankan, yang umumnya terjadi setelah fungsi `setup()` selesai dieksekusi, kendali pemrosesan akan diambil alih oleh sistem operasi untuk menjalankan setiap *task* secara bergantian. Pada kondisi ini, fungsi `loop()` tetap dipertahankan sebagai bagian dari struktur dasar program Arduino, tetapi tidak lagi berperan sebagai pusat logika utama karena fungsinya telah digantikan oleh mekanisme manajemen tugas yang lebih efisien dan terorganisasi di dalam kernel RTOS [5].

3. Apa insight utama yang Anda pelajari dari praktikum ini?

Insight utama dari praktikum ini adalah pemahaman mengenai sinkronisasi antara sensor analog dan aktuator digital secara waktu nyata (*real-time*). Melalui rangkaian tersebut, dipelajari bagaimana mikrokontroler melakukan konversi sinyal analog dari potensiometer menggunakan *Analog to Digital Converter* (ADC) menjadi parameter logika untuk mengendalikan dua LED secara responsif [1], [2].

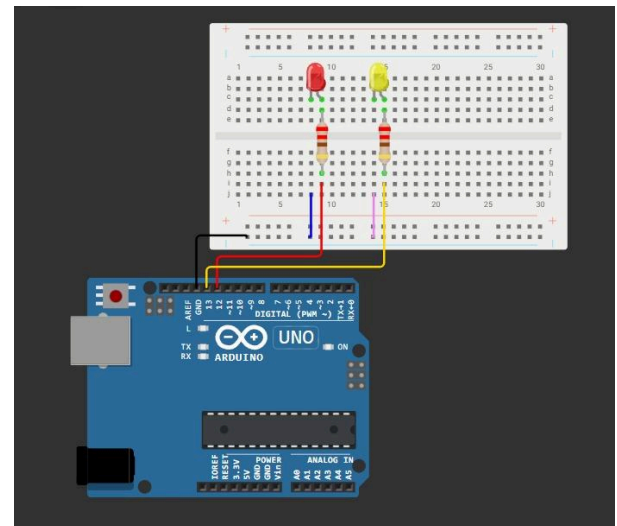
Selain itu, praktikum ini juga memberikan pemahaman penting mengenai stabilitas sistem. Penggunaan ambang batas (*threshold*) dalam kode program menunjukkan bahwa integrasi logika percabangan memiliki peran penting dalam membangun interaksi manusia dan mesin (*human-machine interface*) yang akurat. Kemampuan sistem untuk beralih antara LED merah dan hijau berdasarkan posisi putaran potensiometer membuktikan bahwa pengelolaan masukan dan keluaran (*input-output*) yang tepat merupakan kunci dalam pengembangan perangkat robotika dan otomasi yang andal [3], [4].

Tabel 4-1 Logika dan Pemetaan Pin 5A

Input Analog (Potensiometer)	Nilai ADC (Digital)	Status LED MERAH	Status LED HIJAU
Putar Kiri Maksimal	0	Mati (OFF)	Menyala (ON)

Putar Tengah (Ambang Batas)	512	Menyala (ON)	Mati (OFF)
Putar Kanan Maksimal	1023	Menyala (ON)	Mati (OFF)
Kondisi < 512	0 - 511	Mati (OFF)	Menyala (ON)
Kondisi ≥ 512	512 - 1023	Menyala (ON)	Mati (OFF)

4.2 ANALISIS HASIL PERCOBAAN



Gambar 4-2 Rangkaian Percobaan 2

Berdasarkan hasil pengamatan pada Percobaan 5B, rangkaian yang terdiri atas Arduino Uno dan dua buah *Light-Emitting Diode* (LED) berwarna merah dan kuning menunjukkan bahwa mikrokontroler mampu mengeksekusi instruksi logika digital sekuensial berbasis pewaktuan (*timer*) secara berulang. Ketika program dijalankan, pin digital 11 dan 12 pada Arduino Uno dikonfigurasi sebagai keluaran (*output*) digital untuk mengalirkan arus menuju masing-masing LED melalui resistor pembatas arus pada *breadboard*. Sistem ini bekerja secara bergantian (*flip-flop*), di mana sinyal HIGH dan LOW dikirimkan secara berkala ke masing-masing pin untuk menghasilkan efek visual indikator yang teratur. Hal ini membuktikan kemampuan dasar mikrokontroler dalam mengendalikan komponen keluaran diskrit [2], [3].

Selain itu, pengaturan pergantian kondisi menyala dan mati pada kedua LED menunjukkan bahwa penerapan fungsi penundaan (*delay*) berbasis pewaktuan internal telah berjalan dengan baik. Arduino Uno memanfaatkan osilator kristal bawaan untuk menentukan jeda waktu (misalnya 1000 milidetik atau 1 detik) sebelum melanjutkan ke baris instruksi berikutnya di dalam fungsi

`loop()`. Keberhasilan ini menunjukkan bahwa siklus eksekusi sekuensial pada arsitektur mikrokontroler AVR mampu mengelola pewaktuan dengan tingkat akurasi yang memadai untuk aplikasi kendali digital sederhana, meskipun fungsi `delay()` bersifat *blocking* yang menghentikan sementara eksekusi instruksi lain pada satu inti pemrosesan [4], [5].

Pada kondisi awal (Tahap 1), LED merah yang terhubung ke pin 12 berada pada kondisi HIGH sehingga menyala, sedangkan LED kuning pada pin 11 berada pada kondisi LOW sehingga mati. Sebaliknya, setelah durasi penundaan yang ditentukan, sistem beralih ke Tahap 2 di mana LED kuning menjadi HIGH (menyala) dan LED merah menjadi LOW (mati). Proses transisi yang bergantian ini menunjukkan bahwa logika pemrograman dalam fungsi `loop()` dapat dieksekusi tanpa adanya konflik status maupun tumpang tindih sinyal pada pin keluaran. Hal ini juga menegaskan bahwa penggunaan resistor pembatas arus sangat penting dalam menjaga kestabilan distribusi daya pada jalur *breadboard* agar tidak terjadi lonjakan arus (*overcurrent*) yang dapat merusak pin I/O Arduino [1], [2].

Secara keseluruhan, hasil percobaan ini menunjukkan bahwa pemanfaatan pin digital Arduino Uno untuk mengendalikan beban keluaran digital secara berkala sangat efektif dalam mendukung pemahaman dasar sistem tertanam (*embedded system*). Mekanisme flip-flop sekuensial ini menjadi pondasi penting dalam pengembangan sistem indikator visual, lampu lalu lintas otomatis, maupun sistem peringatan bahaya (*warning system*). Temuan ini menegaskan bahwa ketepatan pewaktuan logika sekuensial serta manajemen arus pada pin I/O merupakan aspek krusial dalam membangun sistem kendali digital yang stabil dan aman [1], [3], [4].

4.2.1 JAWABAN PERTANYAAN PRAKTIKUM PERCOBAAN 2

1. Jelaskan perbedaan antara `loop()` pada Arduino biasa dengan sistem yang menggunakan RTOS!

Pada Arduino standar, fungsi `loop()` bekerja secara sekuensial dan bersifat *blocking*. Artinya, prosesor hanya mengeksekusi satu baris kode pada satu waktu dalam satu alur linier. Apabila terdapat fungsi `delay()`, prosesor akan “tertahan” sehingga tidak dapat menjalankan tugas lain sampai waktu penundaan tersebut selesai.

Sebaliknya, pada sistem yang menggunakan *Real-Time Operating System* (RTOS), eksekusi program dilakukan berdasarkan konsep *multitasking*. RTOS memanfaatkan *scheduler* untuk membagi waktu pemrosesan ke dalam beberapa *task* yang berjalan secara mandiri. Mekanisme ini memungkinkan sistem merespons berbagai kejadian, seperti interupsi atau perubahan status LED, secara *real-time* tanpa terhambat oleh proses penundaan pada bagian kode lainnya [4], [5].

2. Mengapa fungsi `loop()` dibiarkan kosong?

Dalam implementasi RTOS pada lingkungan Arduino, seperti FreeRTOS, fungsi `loop()` umumnya dibiarkan kosong karena kendali alur program telah dialihkan sepenuhnya kepada kernel RTOS. Logika untuk menyalakan LED merah dan kuning didefinisikan di dalam fungsi *task* terpisah yang dibuat menggunakan perintah seperti `xTaskCreate()` pada bagian `setup()`.

Setelah proses inisialisasi selesai, *scheduler* akan mengambil alih kendali CPU untuk menjalankan setiap *task* berdasarkan tingkat prioritasnya. Secara teknis, fungsi `loop()` tetap ada sebagai bagian dari struktur dasar program Arduino, tetapi dalam konteks RTOS fungsi tersebut hanya berperan sebagai *Idle Task* dengan prioritas paling rendah dan tidak lagi digunakan untuk menampung logika utama aplikasi [5].

3. Apa insight utama yang Anda pelajari dari praktikum ini?

Insight utama dari praktikum ini adalah pemahaman yang lebih mendalam mengenai transisi dari pemrograman sekuensial menuju pemrograman berbasis tugas (*task-based programming*). Melalui rangkaian sederhana menggunakan dua LED, dipelajari bahwa sistem tertanam yang stabil memerlukan manajemen waktu (*timing*) yang presisi agar perpindahan logika digital antara pin 11 dan 12 dapat berlangsung tanpa konflik [2], [3].

Selain itu, praktikum ini juga memberikan wawasan bahwa penggunaan RTOS memungkinkan pengembangan sistem yang lebih *scalable* dan responsif. Dengan membagi pengendalian LED ke dalam beberapa *task* independen, sistem dapat ditambahkan fitur lain, seperti pembacaan sensor, tanpa mengganggu pola kedipan LED yang telah berjalan. Hal ini menegaskan bahwa efisiensi pemrosesan dan manajemen sumber daya CPU merupakan faktor utama dalam pengembangan perangkat *Internet of Things* (IoT) maupun sistem robotika yang lebih kompleks [1], [4].

Tabel 4-1 Logika dan Pemetaan Pin 5B

Tahap Operasi	Pin 12 (LED Merah)	Pin 11 (LED Kuning)	Status Fisik LED	Durasi / Jeda Waktu
Tahap 1	HIGH (1)	LOW (0)	LED Merah MENYALA LED Kuning MATI	1 Detik (1000ms)
Tahap 2	LOW (0)	HIGH (1)	LED Merah MATI LED Kuning MENYALA	1 Detik (1000ms)
Kondisi Berulang	Sekuensial	Sekuensial	Bergantian (Flip-Flop)	Terus-menerus
Tahap Operasi	Pin 12 (LED Merah)	Pin 11 (LED Kuning)	Status Fisik LED	Durasi / Jeda Waktu
Tahap 1	HIGH (1)	LOW (0)	LED Merah MENYALA LED Kuning MATI	1 Detik (1000ms)

KESIMPULAN

Berdasarkan praktikum *Real-Time Operating System* (RTOS) yang telah dilaksanakan, dapat disimpulkan bahwa penerapan kernel FreeRTOS pada Arduino Uno menunjukkan adanya perubahan signifikan dari pola pemrograman sekuensial menuju sistem *multitasking* yang lebih terstruktur. Mikrokontroler terbukti mampu berfungsi sebagai pengelola tugas yang efisien, di mana data sensor maupun instruksi kendali dapat diproses secara langsung tanpa penundaan (*buffering*) yang berarti [1], [4].

Hal ini membuktikan bahwa penggunaan RTOS memungkinkan perangkat *embedded system* bekerja secara konsisten baik dari sisi waktu maupun pengelolaan tugas aplikasi [2].

Pada Percobaan 5A (*Multitasking*), penggunaan fungsi *xTaskCreate()* terbukti efektif dalam menjalankan beberapa *task* secara konkuren, seperti pengendalian dua LED dengan interval

berbeda serta proses pencacahan (*counter*). Keberhasilan perpindahan antar-*task* ini menunjukkan bahwa *kernel scheduler* bekerja dengan baik dalam membagi sumber daya CPU melalui mekanisme *context switching* [1], [5]. Selain itu, penggunaan fungsi *vTaskDelay()* terbukti sangat penting dalam menjaga determinisme sistem, di mana kegagalan memenuhi batas waktu (*deadline*) pada sistem *soft RTOS* hanya menyebabkan penurunan performa tanpa merusak sistem secara keseluruhan [2], [4].

Sementara itu, pada Percobaan 5B (*Komunikasi Task*), penggunaan fitur *Queue* menunjukkan tingkat keandalan yang tinggi dalam proses pertukaran data antar-*task* yang berjalan secara terpisah. Mekanisme pengiriman data suhu dan kelembapan melalui fungsi *xQueueSend()* serta penerimaan data menggunakan *xQueueReceive()* memungkinkan distribusi informasi berlangsung secara aman tanpa risiko *race condition* [3], [5]. Proses pengolahan data sensor statis menjadi informasi yang ditampilkan pada *Serial Monitor* menunjukkan bahwa integrasi komunikasi sinkron dalam RTOS mampu menyederhanakan pengelolaan data yang kompleks tanpa mengurangi respons sistem secara *real-time* [3].

Selain itu, keberhasilan implementasi kedua percobaan tersebut sangat dipengaruhi oleh ketepatan konfigurasi perangkat lunak, seperti penentuan ukuran *stack* dan prioritas *task* yang sesuai pada fungsi *xTaskCreate()* [5]. Penggunaan pustaka pendukung juga berperan penting dalam menjaga stabilitas sistem dan memastikan setiap instruksi diproses sesuai jadwal yang telah ditentukan. Kesalahan dalam penentuan prioritas maupun penggunaan fungsi *delay()* standar sebagai pengganti *vTaskDelay()* dapat menyebabkan gangguan pada proses penjadwalan, sehingga ketelitian dalam perancangan sistem menjadi aspek yang sangat penting [1], [4].

Secara keseluruhan, praktikum ini memberikan pemahaman yang komprehensif mengenai arsitektur sistem operasi waktu nyata, khususnya dalam membedakan karakteristik sistem tanpa sistem operasi (*non-OS*) dengan sistem berbasis RTOS yang lebih responsif dan *scalable* [1], [2]. Dengan pemahaman tersebut, praktikan diharapkan mampu mengembangkan sistem kendali dan pemantauan yang lebih kompleks serta efisien untuk diterapkan pada bidang robotika maupun otomasi industri di masa mendatang [2], [3].

DAFTAR PUSTAKA

- [1] LPPM STMIK Handayani, “Implementasi Sistem ...,” *Jurnal Teknologi Informasi*. Tersedia: [Jurnal LPPM STMIK Handayani](#), diakses 15 April 2026.
- [2] ARIMSI, “Polygon Journal ...,” *Polygon Journal*. Tersedia: [Polygon Journal ARIMSI](#), diakses 15 April 2026.
- [3] Universitas Teknologi Digital Indonesia, “JuTI (Jurnal Teknologi Informasi) ...,” *Jurnal Teknologi Informasi (JuTI)*, Yogyakarta, Indonesia. Tersedia: [JuTI UTDI](#), diakses 15 April 2026. (ejournal.akakom.ac.id)
- [4] [DigiKey Electronics](#), “Real-Time Operating Systems and Their Applications,” *DigiKey Articles*, 2021. Tersedia: [RTOS and Their Applications](#), diakses 15 April 2026. ([DigiKey](#))
- [5] [FreeRTOS](#), “xTaskCreate API Reference,” *FreeRTOS Documentation*. Tersedia: [xTaskCreate Documentation](#), diakses 15 April 2026.